
Variational Autoencoders and Learning Disentangled Representations with β -VAE

Syed F Ali

Department of Computer Science
University of California, Irvine
Irvine, CA
sfali1@uci.edu

Sumit Raut

Department of Computer Science
University of California, Irvine
Irvine, CA
sraut@uci.edu

Vishal Srivastava

Department of Computer Science
University of California, Irvine
Irvine, CA
srivasv3@uci.edu

Abstract

By diving into Variational Autoencoders (VAE) and β -VAEs, we explore the effects β has on disentanglement of latent expressions. Using MNIST and dSprites datasets, we evaluate the models impacts on reconstruction quality, and latent space structure. Our results demonstrate that moderate β values promote meaningful disentanglement, balancing the trade-off between fidelity and factorization.

1 Introduction

Variational Autoencoders (VAEs) are generative models capable of learning compressed latent representations of data. Unlike traditional autoencoders, VAEs not only compress and reconstruct data, but also learn continuous probabilistic representations of hidden features. This enables a variety of unique and realistic data samples to be produced resembling the initial data. Despite their success in modeling various data distributions, a standard VAE often resorts to entangled latent spaces. These latent spaces consist of latent dimensions that do not correspond to distinct, interpretable factors varying the data, hindering their capabilities in downstream tasks where isolation of features, and reducing featured dependance, is crucial (i.e. scientific analysis, controllable generation). This calls for the disentanglement of said latent spaces to offer greater transparency and enable efficient control of the generative process.

To address this, the β -VAE framework introduces the hyperparameter β that scales the KL Divergence term in the VAE loss function (more on the KL divergence term in future sections). By increasing the β beyond 1 (original VAE β value), the model is encouraged to compress the latent representation more aggressively. In this project, our aim is to reimplement and evaluate β -VAEs across multiple values of β on datasets with known generative factors, including MNIST, dSprites, and MNIST-Fashion. We investigate the influence of β on the structure of the latent space by analyzing latent traversals qualitatively, measuring reconstruction error quantitatively, and examining the behavior of the resulting KL divergence. We will further analyze the model's learning capabilities by plotting the evolution of the ELBO over each training epoch, shedding light upon the optimizations made by varying the β value. Inspired by capacity scheduling, we also further explore the effect of altering the model's information capacity during the training to better understand how this impacts reconstruction fidelity and disentanglement.

2 Related Works

2.1 Reference 1 : Auto-Encoding Variational Bayes

The foundational Variational Autoencoders (VAEs), by Kingma and Welling [4], described VAEs as a class of deep generative models that use variational inference to learn a probabilistic latent representation, especially when the posterior is intractable and the dataset is large. The main idea is to maximize a variational lower bound (ELBO) on the data likelihood by jointly optimizing a probabilistic encoder and decoder. A crucial part of the paper discusses the importance of the reparameterization trick, introduced to make calculating low variance gradient estimates possible instead of using brute force on the estimated posterior. The paper further describes the application of VAEs in tasks like image generation, denoising, and compression. Our experiments will be working on image data as well, but using its successor, the β -VAE instead.

2.2 Reference 2 : β -VAE: Learning Basic Visual Concepts With A Constrained Variational Framework

Higgins et al. [3] extend the original VAE framework, introduced by Kingma and Welling [4], by incorporating a hyperparameter β into the objective function, scaling the Kullback-Leibler (KL) divergence term. They hypothesize that setting $\beta > 1$ encourages disentangled latent representations and leads to improved qualitative performance compared to the standard VAE. Their results, carried out on the dSprites dataset, that β -VAE can uncover underlying generative factors more effectively than the standard VAE.

2.3 Reference 3 : Understanding disentangling in β -VAE

Burgess et al. [2] further build upon the β -VAE framework by providing an explanation for hyperparameter β 's role in promoting disentanglement. Authors adopted a rate-distortion lens and identified an information bottleneck placed on the latent space using the β -VAE, limiting the amount of information that can be encoded. The constraint on capacity encourages the model to prioritize disentangled, high-variance generative factors over noisy or redundant ones. To improve upon the original β -VAE, the authors introduced capacity scheduling to be able to gradually increase the allowable information capacity in the latent space during training. This modification intends to maintain high reconstruction quality while still achieving disentangled representations.

3 Model Description

3.1 Graphical Model Specification

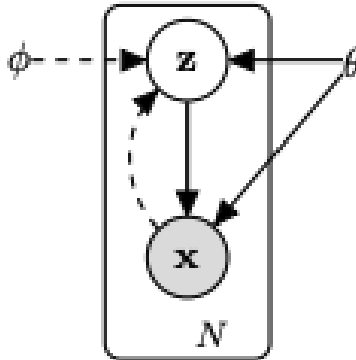


Figure 1: Graphical Model of the VAE

Figure 1 shows the graphical model of the base VAE. It shows the latent variable z that generates the observed data x visualized by a solid arrow $z \rightarrow x$. This generative process is modeled using an encoder/decoder architecture. The decoder, parametrized by θ , defines the likelihood distribution $p_\theta(x | z)$. The encoder, parameterized by ϕ , approximates the true posterior $p(z | x)$, which is generally intractable, using the variational distribution $q_\phi(z | x)$. The dashed arrow from x to z in the figure represents this variational approximation. This process is repeated across a dataset of N samples, indicated by the plate notation with an N in the bottom. The graphical model for the β -VAE remains structurally identical to that of the base VAE.

3.2 Statistical Assumptions

The β -VAE operates on the following key statistical assumptions:

$$p(z) = \mathcal{N}(0, I) \quad (1)$$

The Posterior Distribution shown in Equation 1 assumes the latent variables z are derived from a standard multivariate normal prior (further assuming each latent space is identically and independently distributed).

$$p_\theta(x | z) \quad (2)$$

The Likelihood (Decoder) function shown in Equation 2 defines how the latent variable z generates the observed data x . For binary datasets, such as the MNIST or dSprites, the assumptions are modeled using a Bernoulli distribution. For continuous data, a Gaussian distribution is more appropriate as it assumes the resultant values to be normally distributed.

$$q_\phi(z | x) = \mathcal{N}(\mu_\phi(x), \text{diag}(\sigma_\phi^2(x))) \quad (3)$$

The Posterior Approximation (Encoder) is shown in Equation 3. Due to the true posterior being intractable, an approximation of the variational distribution is conducted. This is done most conveniently using a diagonal Gaussian, as it enables efficient gradient-based optimization using the reparameterization trick.

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (4)$$

The Reparameterization Trick shown in Equation 4 makes it possible to compute the gradients of the estimated posterior without brute force. Since the distribution is parameterized by neural networks outputting $\mu_\phi(x)$ and $\sigma_\phi(x)$, the latent variable z can be expressed as a differentiable transformation of a noise variable $\epsilon \sim \mathcal{N}(0, I)$. This allows gradients to propagate through during training via backpropagation.

3.3 Learning Objective

The main objective of the VAE is to maximize the likelihood of the observed data x by integrating over the latent variables z . To do this, VAE maximizes the tractable lower bound referred to as the Evidence Lower Bound (ELBO) instead of the actual marginal likelihood $p(x) = \int p(x | z)p(z)dz$ since it is usually intractable. The ELBO Objective is given by:

$$\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x | z)] - \text{KL}(q_\phi(z | x) \| p(z)) \quad (5)$$

The ELBO objective consists of two key terms, the reconstruction term and the regularization (KL) term. The reconstruction term encourages accurate generation of x given latent samples z . The regularization term minimizes the divergence between the approximate posterior $q_\phi(z | x)$ and the prior $p(z)$. This regularization term is modified in the β -VAE implementation by introducing a weighting coefficient ($\beta > 1$) making the ELBO objective:

$$\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x | z)] - \beta \cdot \text{KL}(q_\phi(z | x) \| p(z)) \quad (6)$$

This enables more control over the trade-off between reconstruction fidelity and latent complexity. Increasing β value pushes the $q_\phi(z | x)$ towards the prior $p(z)$, instructing the model to compress information in the latent space. This results in more interpretable and disentangled representations at the cost of reconstruction fidelity.

4 Inference and Learning

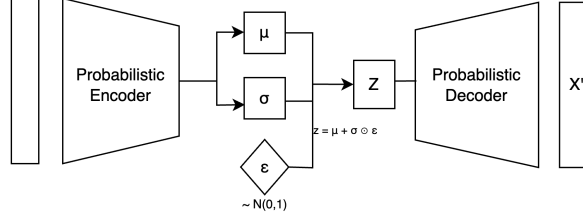


Figure 2: Model of the VAE

4.1 Inference Algorithm

In the β -VAE framework, inference involves estimating the intractable posterior $p(z | x)$ over the latent variable z given observed data x . This is addressed by introducing a variational approximation $q_\phi(z | x)$, parameterized by an encoder network.

To enable gradient-based optimization, the reparameterization trick is employed. Instead of sampling $z \sim q_\phi(z | x)$ directly, we reparameterize as $z = \mu_\phi(x) + \sigma_\phi(x) \cdot \epsilon$, where $\epsilon \sim \mathcal{N}(0, I)$. This allows gradients to propagate through the sampling operation, making it feasible to optimize the encoder parameters ϕ via stochastic gradient descent.

During inference, the encoder produces a distribution over latent codes z , which are then used by the decoder $p_\theta(x | z)$ to reconstruct the input. This process is trained end-to-end using the Stochastic Gradient Variational Bayes (SGVB) estimator.

4.2 Derivation of the Objective Function

We begin from the marginal log-likelihood of a single datum $\mathbf{x}$ under our generative model:

$$\log p_\theta(\mathbf{x}) = \log \int p_\theta(\mathbf{x}, \mathbf{z}) d\mathbf{z} = \log \int q_\phi(\mathbf{z} | \mathbf{x}) \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z} | \mathbf{x})} d\mathbf{z}. \quad (7)$$

Applying Jensen's inequality to the logarithm yields the evidence lower bound (ELBO):

$$\log p_\theta(\mathbf{x}) \geq \int q_\phi(\mathbf{z} | \mathbf{x}) \log \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z} | \mathbf{x})} d\mathbf{z} \equiv \mathcal{L}(\theta, \phi; \mathbf{x}) = \mathbb{E}_{q_\phi(\mathbf{z} | \mathbf{x})} [\log p_\theta(\mathbf{x} | \mathbf{z})] - \text{KL}(q_\phi(\mathbf{z} | \mathbf{x}) \| p(\mathbf{z})). \quad (8)$$

In the β -VAE framework,

For a minibatch $\{\mathbf{x}^{(i)}\}_{i=1}^B$, we maximize the weighted evidence lower bound

$$\mathcal{L}_\beta = \frac{1}{B} \sum_{i=1}^B \left[\mathbb{E}_{q_\phi(\mathbf{z} | \mathbf{x}^{(i)})} [\log p_\theta(\mathbf{x}^{(i)} | \mathbf{z})] - \beta \text{KL}(q_\phi(\mathbf{z} | \mathbf{x}^{(i)}) \| p(\mathbf{z})) \right].$$

Our project implements a scalar weight $\beta > 0$ on the KL divergence, yielding the final training objective

$$\mathcal{L}_\beta(\theta, \phi; \mathbf{x}) = \mathbb{E}_{q_\phi(\mathbf{z} | \mathbf{x})} [\log p_\theta(\mathbf{x} | \mathbf{z})] - \beta \cdot \text{KL}(q_\phi(\mathbf{z} | \mathbf{x}) \| p(\mathbf{z})). \quad (9)$$

We approximate the intractable posterior with a Gaussian encoder

$$q_\phi(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\mathbf{z}; \mu_\phi(\mathbf{x}), \text{diag}(\exp(\log \sigma_\phi^2(\mathbf{x})))),$$

where $\mu_\phi(\mathbf{x})$ and $\log \sigma_\phi^2(\mathbf{x})$ are outputs of an encoder network with parameters ϕ .

and for a Gaussian decoder,

$$\log p_\theta(\mathbf{x} \mid \mathbf{z}) = -\frac{1}{2} \sum_i \left[\frac{(x_i - \mu_i)^2}{\sigma_i^2} + \log(2\pi\sigma_i^2) \right].$$

For a Bernoulli decoder,

$$\log p_\theta(\mathbf{x} \mid \mathbf{z}) = \sum_i \left[x_i \log \hat{x}_i + (1 - x_i) \log(1 - \hat{x}_i) \right],$$

Maximizing $\mathcal{L}_\beta$ with respect to θ, ϕ via stochastic gradient ascent constitutes the training algorithm.

4.3 Optimization Details

All models were implemented in PyTorch and trained by stochastic gradient ascent on the β -VAE ELBO. Key settings and hyperparameters are:

- **Reparameterization trick:**

$$\mathbf{z} = \mu_\phi(\mathbf{x}) + \exp\left(\frac{1}{2} \log \sigma_\phi^2(\mathbf{x})\right) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

- **Optimizer:** Adagrad with learning rate 1×10^{-2} and default accumulator 10^{-6} .

- **Batch sizes:**

- MNIST : 128
- dSprites : 128

- **Number of epochs:**

- Preliminary experiments: 3–5 epochs
- Final runs: 30(MNIST) and 50(dSprite) epochs with observations after every 10 epochs with disentanglement matrices values for dSprite.

- **Capacity scheduling:** Gradually ramp the effective KL weight from 0 to β over T_c epochs (approach mentioned in [2]):

$$C_t = \min\left(\frac{t}{T_c}, 1\right) \cdot \beta, \quad \text{and replace } \beta \cdot \text{KL} \mapsto |\text{KL} - C_t|.$$

- **Logging and checkpointing:**

- Per-epoch losses (total, recon, KL) printed to console and saved to `results/train.log`.
- Held-out ELBO and disentanglement metrics computed every `eval_freq` epochs.
- Model checkpoints saved to `results/vae_Nz_{...}.pth`.

5 Implementation

5.1 Codebase Overview

Our codebase contains a PyTorch implementation of the Beta-Variational Autoencoder (VAE), based on the paper Auto-Encoding Variational Bayes by Diederik P. Kingma and Max Welling. The codebase is structured to facilitate training, evaluation, and experimentation with VAEs.

The core components of the project are organized as follows:

- **Main Files:**

- `main.py`: The primary script for running experiments, including training and evaluation of the VAE.
 - `metrics.py`: Contains functions for calculating and reporting various performance metrics.
 - `plot_metrics.py`: Used for visualizing the metrics obtained during training and evaluation.
 - `vae_net.py`: Defines the architecture and components of the Variational Autoencoder neural network.
- **Directories:**
 - `data/`: stores raw or processed datasets used for training.
 - `results/`: Stores the output of experiments, such as generated images, plots, or statistical results.
 - `saved_models/`: Used to save trained VAE model weights and configurations.
 - `utils/`: Contains utility functions and helper scripts shared across different parts of the project.

The repository supports training VAEs on various datasets, including MNIST, Dsprites, and Fashion-MNIST Face, and provides functionality for generating reports based on the experimental results.

For more details, visit the repository on GitHub: [Beta_Variational_Auto_Encoder](#)

5.2 Libraries and Tools

Our experiments were implemented in Python using the PyTorch deep learning framework. We used the following libraries and tools:

- **PyTorch (v2.1)**: Core framework for model construction, training, and backpropagation.
- **Data loader**: For loading and preprocessing the MNIST dataset.
- **NumPy** and **SciPy**: For numerical computations and statistical utilities.
- **Matplotlib**: Used for plotting training curves and latent space traversals.
- **argparse**: To enable flexible configuration via command-line arguments.
- **Pillow (PIL)**: For image handling (used indirectly through `torchvision`).

All experiments were conducted using GPUs to accelerate training. The codebase supports modular experimentation across different VAE variants, β -schedules, and decoder architectures.

5.3 Dataset Description

We evaluated our model’s performance on two distinct image datasets: MNIST and dSprites.

MNIST is a widely used benchmark dataset for image classification. It consists of a large collection of handwritten digit images, specifically 28×28 grayscale images of digits from 0 to 9. The dataset is split into 60,000 training images and 10,000 test images. MNIST is commonly employed for tasks involving digit recognition and serves as a fundamental testbed for new machine learning models due to its simplicity and well-defined structure.

dSprites is a dataset of synthetic 2D shapes designed specifically for research on disentangled representations. It comprises 64×64 grayscale images of simple shapes (square, ellipse, heart) that vary along five independent, known latent factors: shape, scale, rotation, and X/Y positions. The explicit ground-truth generative factors make dSprites an invaluable benchmark for quantitatively evaluating a model’s ability to learn disentangled latent representations, where each latent dimension ideally corresponds to a single, interpretable generative factor.

6 Evaluation and Results

6.1 Evaluation Protocols

To assess how β affects the trade-off between reconstruction quality and disentanglement in Variational Autoencoders (VAEs), we used both **quantitative** metrics and **qualitative visualizations** across multiple datasets.

Quantitative Metrics:

- **Reconstruction Loss:** Binary cross-entropy (BCE) between the original and reconstructed image.
- **KL Divergence:** Measures divergence of the learned posterior from the standard normal prior.
- **ELBO (Evidence Lower Bound):** Computed as the negative sum of reconstruction loss and scaled KL divergence (controlled by β). This serves as a proxy for model likelihood.

Qualitative Metrics. We conducted latent space traversals by varying a single latent dimension while keeping others fixed, and visualized how output images changed. Successful disentanglement is evidenced by clean, interpretable transformations corresponding to known generative factors.

6.2 Training Curves

We trained models on MNIST and dSprites for multiple values of β (e.g., $\beta = 2, 4, 8, 16$). The following patterns were consistently observed:

- **ELBO vs. Epoch:** ELBO increased over epochs for all β values, indicating that the model progressively learned to better approximate the true data distribution.
- **KL Divergence vs. Epoch:** For larger β values, KL divergence was lower at convergence, consistent with the stronger regularization imposed on the latent space.
- **Reconstruction Loss:** Higher β led to worse reconstruction, reflecting the trade-off between compression and fidelity.

Parameter	dSprites	MNIST
Dataset	dSprites	MNIST
Epochs	50	30
Batch size	128	128
β (KL weight)	2.0,4.0,8.0,16.0	2.0,4.0,8.0,16.0
Decoder type	Bernoulli	Bernoulli
Latent dimension	20	10

Table 1: Training parameters used for β -VAE on dSprites and MNIST datasets.

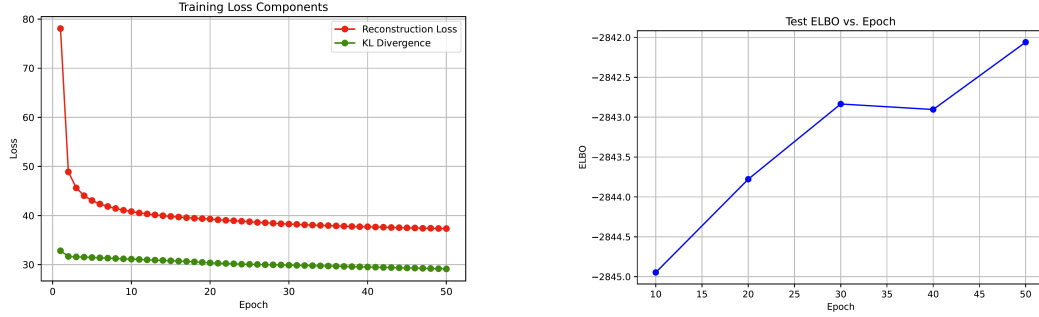


Figure 3: $\beta = 2$ is too weak to fully disentangle—the model overfits to reconstruction

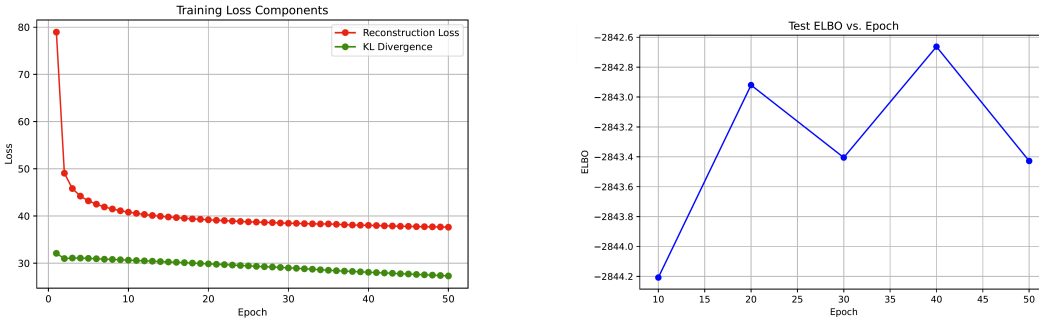


Figure 4: $\beta = 4$ strikes the ideal balance: it achieves significant ELBO gain (best test likelihood), while still yielding crisp reconstructions and encouraging each latent dimension to specialize (as seen in the traversal grids), So for dSprites, $\beta = 4$ is the best choice.

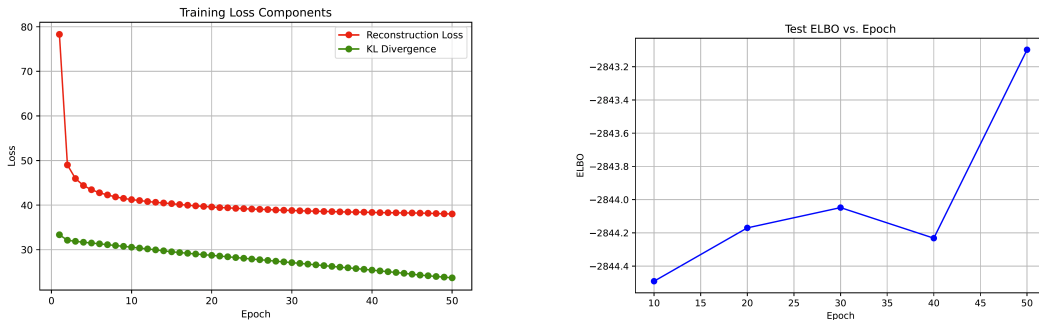


Figure 5: $\beta = 8$ over-regularize—latent usage collapses, reconstructions blur, and ELBO hardly improves

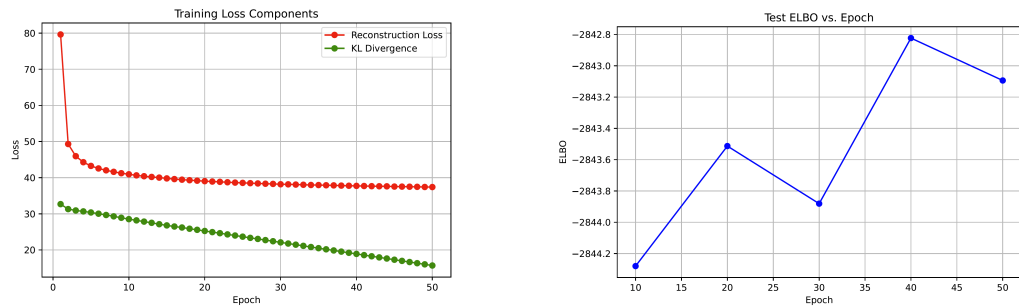


Figure 6: $\beta = 16$ over-regularize—latent usage collapses, reconstructions blur, and ELBO hardly improves

Figure 7: Training curves for different β values on dSprites dataset. Each row shows KL + reconstruction loss (left) and test ELBO (right). Higher β values increase regularization strength.

6.3 Qualitative Visualizations

We generated grids of reconstructed images from latent traversals. For $\beta = 1$, we observed entangled transformations—changes in one latent variable affected multiple visual aspects (e.g., shape and size in dSprites). In contrast, with $\beta = 4$ or $\beta = 8$, latent dimensions controlled more distinct factors of variation, such as:

- **dSprites:** scale, orientation, x/y position.
- **MNIST:** stroke thickness, digit slant, and style.

This behavior aligns with the information bottleneck principle: increasing β forces the model to compress representations and disentangle information into separate factors.

6.4 Interpretation of Results

Our results confirm the core hypothesis of β -VAE:

- **Lower β** (e.g., $\beta = 1$) prioritizes reconstruction fidelity but often results in entangled representations.
- **Moderate β** (e.g., $\beta = 4$) strikes a balance and produces latent variables aligned with interpretable factors.
- **Higher β** (e.g., $\beta = 8, 16$) enhances disentanglement at the cost of reconstruction quality, often producing blurry or lossy outputs.

The qualitative visualizations and divergence trends confirm that β indeed governs the trade-off between reconstruction accuracy and the structure of the learned latent space.

Table 2: An overview of how the hyperparameter β influences the latent space and generative traversals in a β -VAE. Adjusting β controls the trade-off between reconstruction quality and the disentanglement of latent factors.

β Value	Effect on Latents	Effect on Traversals
$\beta \approx 1$ (Standard VAE)	A balanced trade-off where latents encode useful information while remaining close to the Gaussian prior.	Traversals typically show smooth changes, but latent dimensions are often entangled—modifying one latent can alter multiple, distinct visual attributes.
$\beta < 1$	Less pressure on the KL divergence gives the model more freedom to encode detailed information, moving the posterior further from the prior.	Latents can overfit the training data. Traversals might exhibit sharp, detailed changes, but these are generally highly entangled.
$\beta > 1$	The stronger KL penalty encourages disentanglement, forcing the model to learn a more compressed and structured representation.	Traversals often become cleaner and more interpretable. If successful, each latent dimension corresponds to a single factor of variation.
$\beta \gg 1$ (Over-regularized)	The model may underfit because the strong regularization outweighs the reconstruction loss, leading to a lossy encoding that discards critical information.	Traversals may show minimal variation or produce corrupted images, as the latent space has failed to capture the essential data-generating factors.

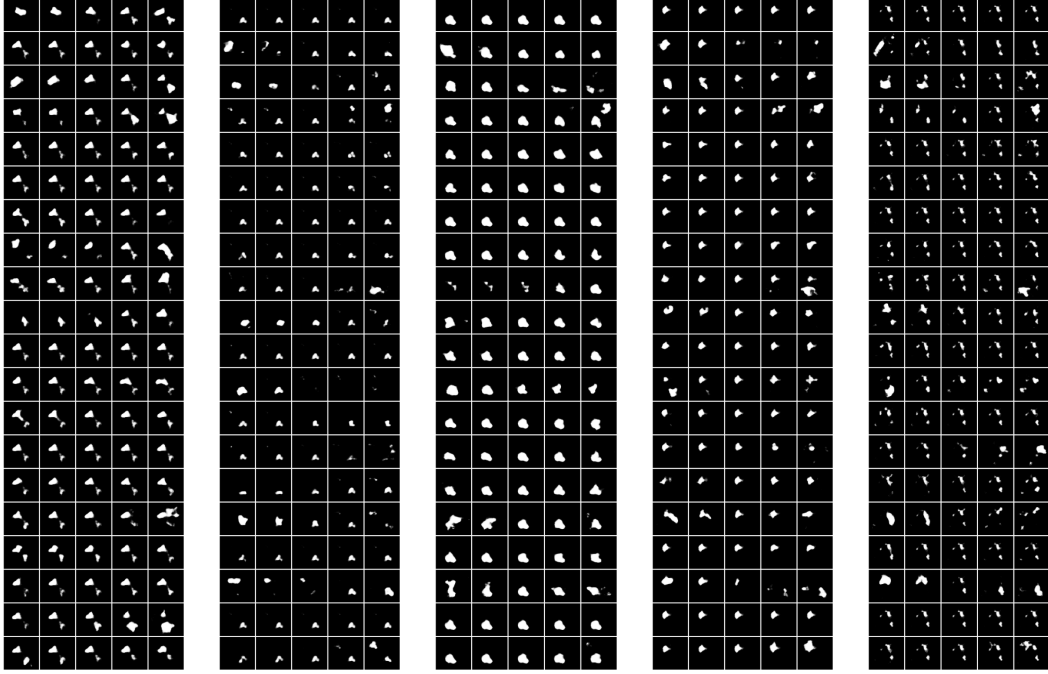


Figure 8: Latent Traversals for $\beta = 4$ at 10, 20, 30, 40 and 50 epochs respectively

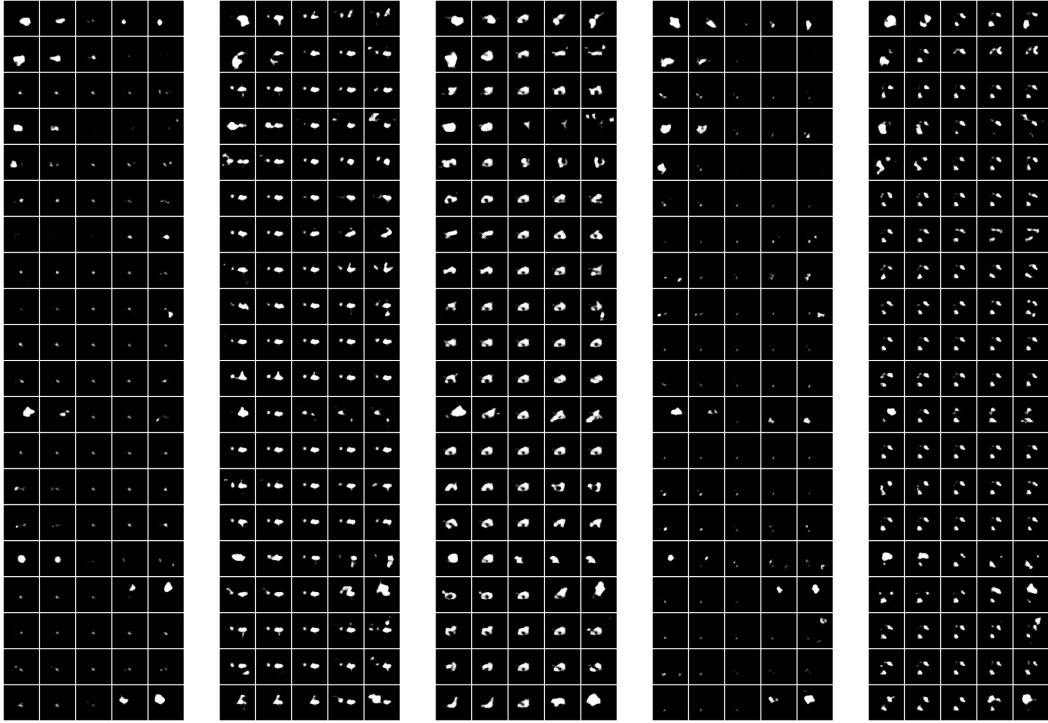


Figure 9: Latent Traversals for $\beta = 8$ at 10, 20, 30, 40 and 50 epochs respectively

Figure 10: Latent space traversals for different β values across training epochs for dSprites. Each subplot varies a single latent dimension while others are fixed, demonstrating the evolution of disentanglement. For $\beta = 4$, by epoch 10 we see clear, coherent variations in position and scale, indicating the model is effectively using its latent capacity early on. For $\beta = 8$, early in training (epochs 10–20), most latents remain “off” and reconstructions are noticeably blurrier, as the heavier KL penalty forces the model to under-utilize its code. Empirically, $\beta = 4$ offers the best trade-off: it achieves strong disentanglement nearly as cleanly as $\beta = 8$, yet maintains far sharper reconstructions, making it the preferable setting for dSprites in this project.

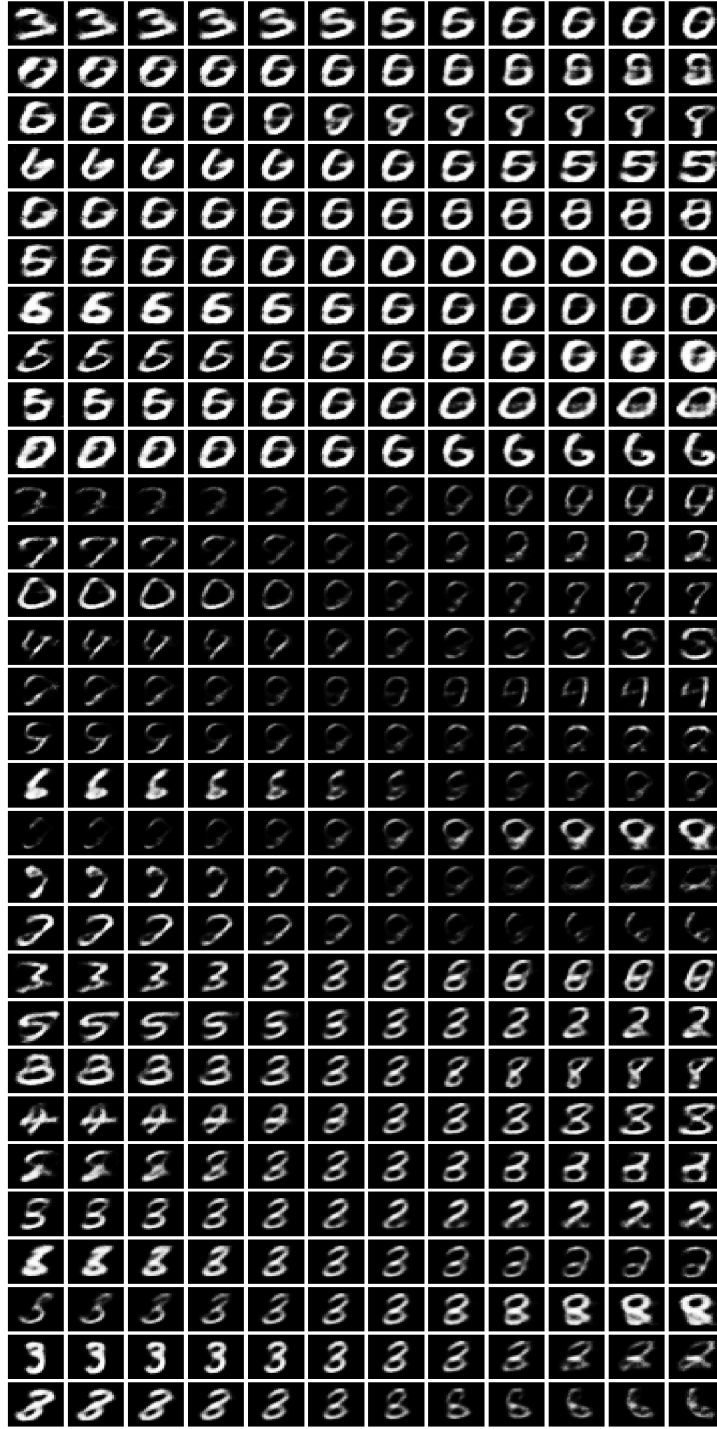


Figure 11: **Latent Traversals on MNIST using a β -VAE ($\beta = 4.0$, $N_z = 10$).** This visualization illustrates how individual latent dimensions z_i ($i = 0, 1, \dots, 9$) influence the generated output. For each latent variable, 3 different latent vectors (random seeds) are sampled from $\mathcal{N}(0, I)$, and the i -th coordinate is varied linearly from -3 to $+3$ across 12 steps while keeping other dimensions fixed. Each group of three rows corresponds to a single z_i ; each row within the group uses a different seed. Columns show the effect of changing the latent dimension, while rows show this effect across different starting points. If a latent dimension is disentangled and meaningful, modifying it causes consistent and interpretable changes (e.g., stroke width, rotation, or digit type) across different seeds.

7 Discussion

7.1 Limitations

Applying VAEs to image datasets like dSprites presents several limitations, particularly in achieving disentangled and high-fidelity outputs. A significant challenge with β -VAEs is the delicate balance required between achieving accurate image reconstructions and learning disentangled latent factors, a trade-off that is highly sensitive to the chosen value and scheduling of the hyperparameter β . The model can also suffer from posterior collapse, where the latent variables are ignored if the decoder is excessively strong, leading to impoverished latent representations. Furthermore, finding the optimal β value is a practical challenge. While a common range for β is $[1, 100]$, "sweet spots" like $\beta = 4$ as used by [3] for dSprites or values between 5–10 used by subsequent works (e.g., [2]) for balancing reconstruction and factorization are frequently observed. However, values of $\beta > 10$ can lead to more independent latent factors at the cost of blurrier reconstructions.

7.2 Future Work

Future work may focus on areas like **a) Architectural Inductive Biases**—Our current MLP-based β -VAE treats each pixel independently, missing the inherent spatial structure of images. We plan to implement and compare a Conv- β -VAE against our MLP baseline on MNIST, Fashion-MNIST and dSprites using ELBO, reconstruction error, β -VAE score, MIG, and traversal grids. **b) Cross-Dataset Generalization and Transfer** Having validated disentangling on synthetic dSprites, we will assess whether those learned latent axes transfer to related but distinct datasets—such as dSprites $\rightarrow$ 3D-Shapes $\rightarrow$ Fashion-MNIST. Demonstrating latent-factor reuse across domains would show that our β -VAE not only disentangles within a dataset but also captures semantically meaningful features that generalize.

8 Conclusion

Our investigation into β -VAEs confirms that the hyperparameter β is crucial in balancing reconstruction quality and latent space disentanglement. Moderate β values, such as $\beta = 4$, promote meaningful disentanglement, allowing latent dimensions to correspond to distinct, interpretable factors of variation in datasets like dSprites and MNIST. However, increasing β too much (e.g., $\beta = 8, 16$) leads to over-regularization, where disentanglement is enhanced at the cost of reconstruction fidelity, resulting in blurrier outputs. Conversely, lower β values prioritize reconstruction but often lead to entangled representations. This shows the importance of carefully selecting β to achieve an optimal trade-off in learning disentangled and interpretable latent representations.

9 Team Contributions

- **Vishal Srivastava (srivasv3@uci.edu)**
VAE implementation and refactoring of the original [1] MLP-based encoder/decoder into our `vae_net.py` module. Designed and coded the unified `argparse` interface in `main.py`, added support for both Bernoulli and Gaussian decoders, and integrated the Adam optimizer with proper weight initializations and variance clamping to avoid NaNs.
- **Sumit Raut (scraut@uci.edu)**
Extended the data-loading to include dSprites, MNIST, FashionMNIST: writing the `DSpritesDataset` class, implemented in PyTorch's data loader, and ensuring batching for datasets. Also implemented the latent-traversal visualization (`generate_latent_traversals`) and automated checkpoint/log management.
- **Syed F Ali (sfali1@uci.edu)**
Developed the disentanglement evaluation suite: authored `metrics.py` (β -VAE score & MIG) and `plot_metrics.py` for curve generation, and debugged the sample-lookup logic to handle the full dSprites factor set. Wrote the `utils/test.py` ELBO evaluator, implemented capacity-scheduling logic, and drafted the report sections.

Following these technical contributions, the preparation of project reports and proposals was a collaborative effort with equal contributions from all three members.

References

- [1] Zhifan Ang. Auto-Encoding Variational Bayes. https://github.com/angzhifan/Auto-Encoding_Variational_Bayes, 2020. GitHub repository.
- [2] Christopher P Burgess, Irina Higgins, Arka Pal, Loic Matthey, Nicholas Watters, Guillaume Desjardins, and Alexander Lerchner. Understanding disentangling in β -vae. *arXiv preprint arXiv:1804.03599*, 2018.
- [3] Irina Higgins, Loic Matthey, Arka Pal, Christopher Burgess, Xavier Glorot, Matthew Botvinick, Shakir Mohamed, and Alexander Lerchner. beta-vae: Learning basic visual concepts with a constrained variational framework. In *International Conference on Learning Representations (ICLR)*, 2017.
- [4] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.